Automatic Differentiation

Benoît Legat

☐ Full Width Mode ☐ Present Mode

Table of Contents

Differentiation approaches

Chain rule

Forward Differentiation

Reverse Differentiation

Comparison

Discontinuity

Neural network

Wine example 🍷

Second-order

Differentiation approaches

We can compute partial derivatives in different ways:

1. **Symbolically**, by fixing one of the variables and differentiating with respect to the others, either manually or using a computer.
2. **Numerically**, using the formula $f'(x) \approx (f(x+h) - f(x))/h$.
3. **Algorithmically**, either forward or reverse : this is what we will explore here.

Chain rule

Consider $f(x) = f_3(f_2(f_1(x)))$. If we don't have the expression of f_1 but we can only evaluate $f_i(x)$ or $f'(x)$ for a given x ? The chain rule gives

$$f'(x) = f_3'(f_2(f_1(x))) \cdot f_2'(f_1(x)) \cdot f_1'(x).$$

Let's define $s_0 = x$ and $s_k = f_k(s_{k-1})$, we now have:

$$f'(x) = f_3'(s_2) \cdot f_2'(s_1) \cdot f_1'(s_0).$$

Two choices here:

Forward	Reverse
$t_0 = 1$	$r_3 = 1$
$t_k = f_k'(s_{k-1}) \cdot t_{k-1}$	$r_k = r_{k+1} \cdot f_{k+1}'(s_k)$

Forward Differentiation

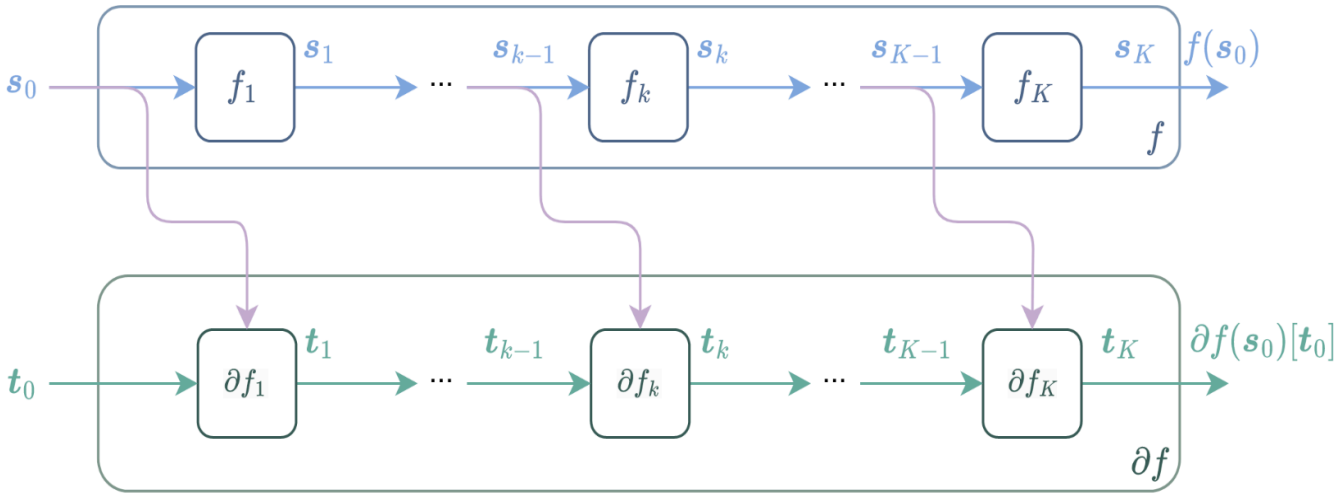


Figure 8.1

Implementation

```
struct Dual{T}
    value::T # s_k
    derivative::T # t_k
end
```

```
Base.:-(x::Dual{T}) where {T} = Dual(-x.value, -x.derivative)
```

```
Base.:*(x::Dual{T}, y::Dual{T}) where {T} = Dual(x.value * y.value, x.value * y.derivative + x.derivative * y.value)
```

```
Dual(-3, -10)
```

```
-Dual(1, 2) * Dual(3, 4)
```

```
f_1 (generic function with 1 method)
```

```
f_1(x, y) = x * y
```

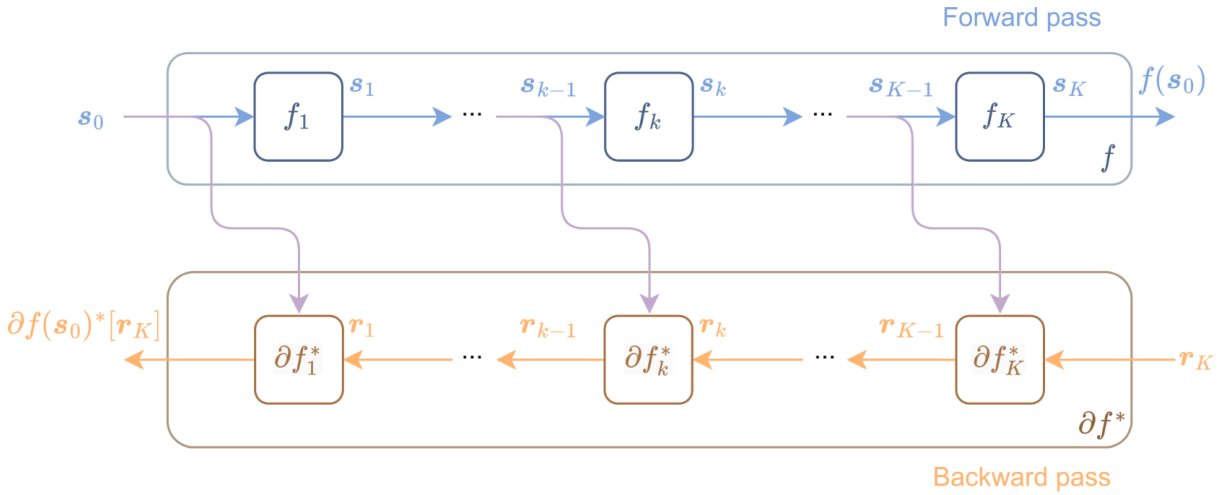
```
f_2 (generic function with 1 method)
```

```
f_2(s1) = -s1
```

```
Dual(-3, -10)
```

```
(f_2 ∘ f_1)(Dual(1, 2), Dual(3, 4))
```

Reverse Differentiation



Two different takes on the multivariate chain rule

The chain rule gives us

$$\frac{\partial f_3}{\partial x}(f_1(x), f_2(x)) = \partial_1 f_3(s_1, s_2) \cdot \frac{\partial s_1}{\partial x} + \partial_2 f_3(s_1, s_2) \cdot \frac{\partial s_2}{\partial x}$$

To compute this expression, we need the values of $s_1(x)$ and $s_2(x)$ as well as the derivatives $\partial s_1/\partial x$ and $\partial s_2/\partial x$.

Common to forward and reverse: Given s_1, s_2, computes **local** derivatives $\partial_1 f_3(s_1, s_2)$ and $\partial_2 f_3(s_1, s_2)$, shortened $\partial_1 f_3, \partial_2 f_3$ for conciseness.

Forward

$$\begin{aligned} t_3 &= \partial_1 f_3 \cdot t_1 + \partial_2 f_3 \cdot t_2 \\ &= \begin{bmatrix} \partial_1 f_3 & \partial_2 f_3 \end{bmatrix} \cdot \begin{bmatrix} t_1 \\ t_2 \end{bmatrix} \\ &= \partial f_3 \cdot \begin{bmatrix} t_1 \\ t_2 \end{bmatrix} \end{aligned}$$

Reverse

$$\begin{aligned} \begin{bmatrix} r_1 & r_2 \end{bmatrix} &\mathrel{+}= r_3 \cdot \partial f_3 \\ &\mathrel{+}= r_3 \cdot \begin{bmatrix} \partial_1 f_3 & \partial_2 f_3 \end{bmatrix} \\ &\mathrel{+}= \begin{bmatrix} r_3 \cdot \partial_1 f_3 & r_3 \cdot \partial_2 f_3 \end{bmatrix} \end{aligned}$$

Reverse*

$$\begin{aligned} \begin{bmatrix} r_1 \\ r_2 \end{bmatrix} &\mathrel{+}= \partial f_3^* \cdot r_3 \\ &\mathrel{+}= \begin{bmatrix} \partial_1 f_3 \\ \partial_2 f_3 \end{bmatrix} \cdot r_3 \\ &\mathrel{+}= \begin{bmatrix} \partial_1 f_3 \cdot r_3 \\ \partial_2 f_3 \cdot r_3 \end{bmatrix} \end{aligned}$$

When using automatic differentiation, don't forget that we must always evaluate the derivatives. For the following example we choose to evaluate it in $x = 3$

Error message

```
InterruptException:
```

```
qa(md"Apply the automatic differentiation to ``s_3=f_3(s_1, s_2) = s_1 + s_2``, with ``s_1=f_1(x) = x`` and ``s_2=f_2(x) = x^2``",
hbox([
    md"""
#### Forward

* ``\partial x / \partial x = 1``
* ``\partial s_1 / \partial x = 1 \vert_{x=3} \cdot 1 = 1``
* ``\partial s_2 / \partial x = 2x \vert_{x=3} \cdot 1 = 6``
* ``\partial s_3 / \partial x = 1 \vert_{x=3} \cdot 1 + 1 \vert_{x=3} \cdot 6 = 7``
""",
    Div(html" ", style = Dict("flex-grow" => "1")),
md"""
#### Reverse

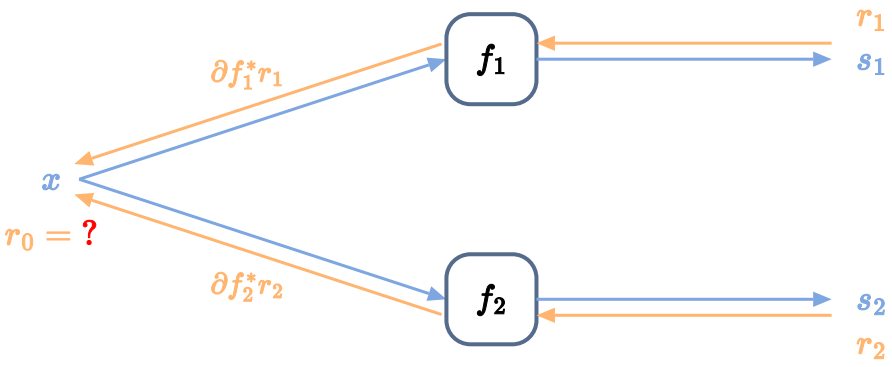
* Initialize ``\partial s_3 / \partial s_1 = \partial s_3 / \partial s_2 = \partial s_3 / \partial x = 0``
* First part: ``\partial s_3/\partial s_1 \mathrel{\raise{0.19ex}{\scriptstyle+}} = 1``
  - ``\partial s_3 / \partial x \mathrel{\raise{0.19ex}{\scriptstyle+}} = 1 \cdot 1 \vert_{x=3}``
* Second part: ``\partial s_3/\partial s_2 \mathrel{\raise{0.19ex}{\scriptstyle+}} = 1``
  - ``\partial s_3 / \partial x \mathrel{\raise{0.19ex}{\scriptstyle+}} = 1 \cdot 2x \vert_{x=3}``
* The result is ``\partial s_3 / \partial x = 7``.
"""]))
```

👀 Reading hidden code

Reverse mode on a DAG

- Forward mode: we know how to propagate t through any graph.
- Reverse mode on a chain: we know that $r_{k-1} = \partial f_k^* r_k$.
- Reverse mode on a DAG: ❓

Consider the simplest DAG that is not a chain: x is used by both f_1 and f_2. The reverse pass brings two contributions $\partial f_1^* r_1$ and $\partial f_2^* r_2$ back to the same x.



Error message

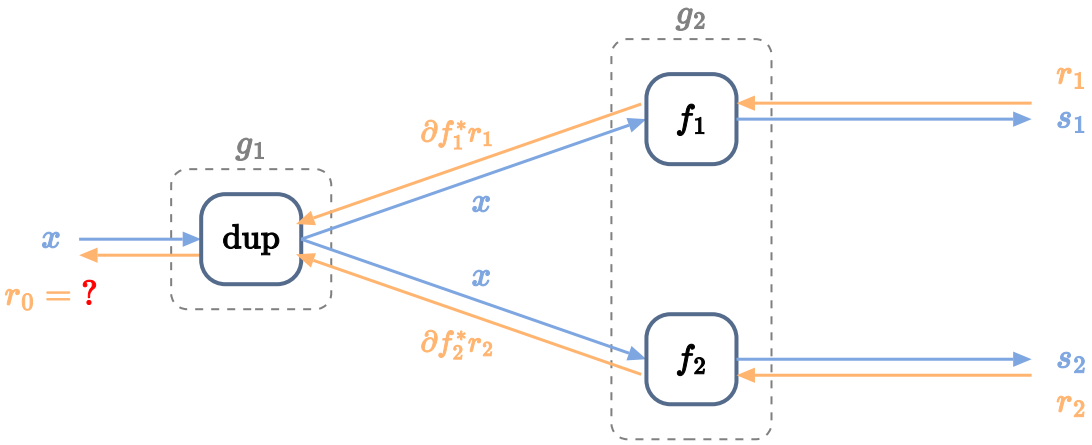
```
InterruptException:
```

```
1 qa(md"How should we combine ``\partial f_1^* r_1`` and ``\partial f_2^* r_2`` into
  ``r_0`` ? Sum them ? Multiply them ?", md"We don't know yet! 😱 The only way to
  find out is to make sure that reverse mode computes the same derivative as forward
  mode. But we only know how to do this for a **chain**... and this is not a chain.")
```

👀 *Reading hidden code*

Let's turn it into a chain!

Key idea: introduce $\mathbf{dup}(x) = (x, x)$ so that $f = g_2 \circ g_1$ is a chain with $g_1 = \mathbf{dup}$ and $g_2(x_1, x_2) = (f_1(x_1), f_2(x_2))$.



f_1 and f_2 now contribute to **different** entries of the output of **dup** so there is nothing to combine:

$$\partial g_2^* \begin{bmatrix} r_1 \\ r_2 \end{bmatrix} = \begin{bmatrix} \partial f_1^* & 0 \\ 0 & \partial f_2^* \end{bmatrix} \begin{bmatrix} r_1 \\ r_2 \end{bmatrix} = \begin{bmatrix} \partial f_1^* r_1 \\ \partial f_2^* r_2 \end{bmatrix}$$

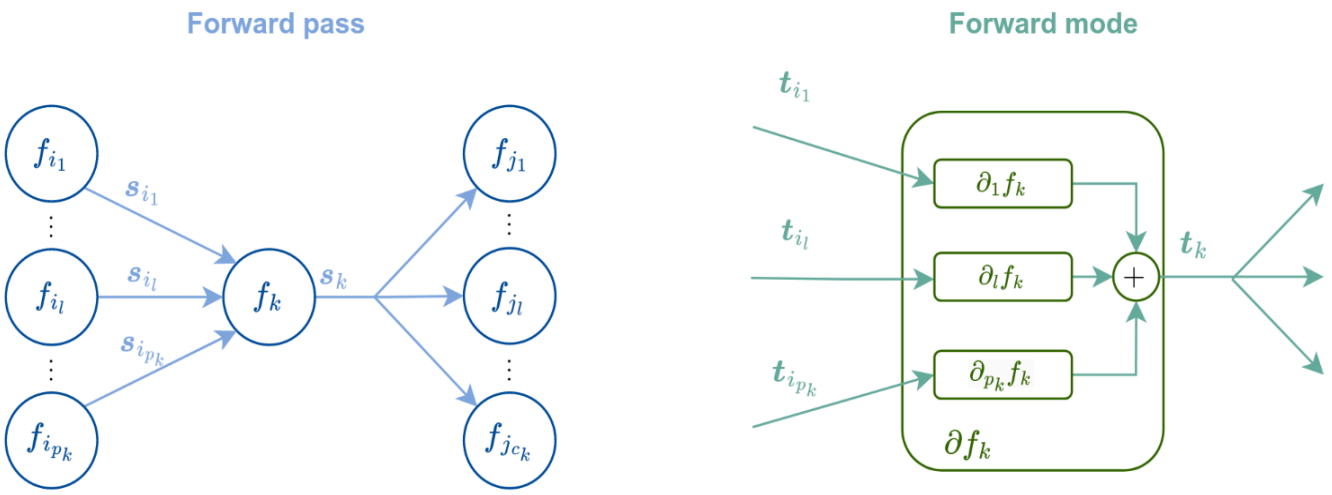
What remains is the reverse pass of **dup**... but **dup** is a function like any other: we can compute its Jacobian!

Error message

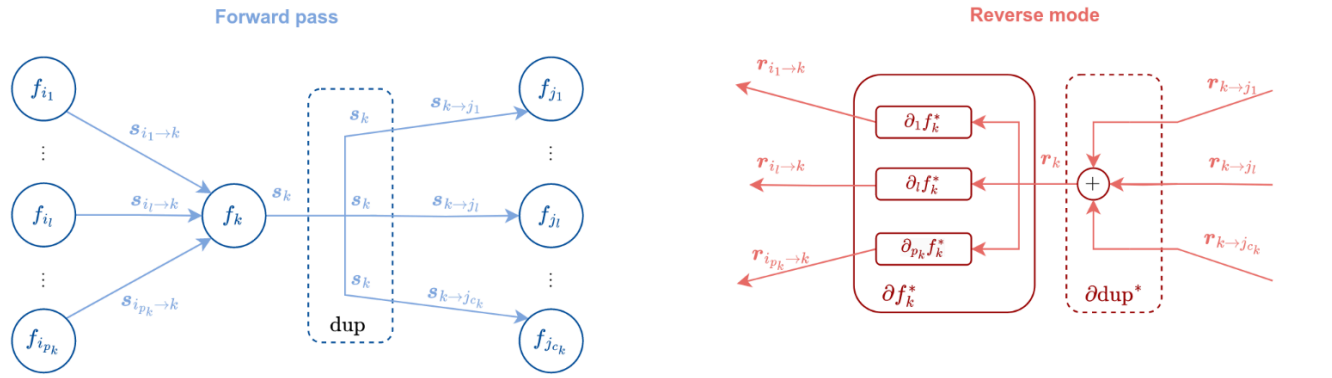
```
InterruptException:
```

```
qa(md"What is the reverse pass of ``\text{dup}`` ?", md"""
In forward mode, ``\partial \text{dup} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}`` maps ``t`` to ``(t, t)``.
For the reverse pass to compute the same derivative, it must multiply by its adjoint ``\partial \text{dup}^* = \begin{bmatrix} 1 & 1 \end{bmatrix}``:
```math
r_0
=
\begin{bmatrix} 1 & 1 \end{bmatrix}
\begin{bmatrix} \partial f_1^* r_1 \\ \partial f_2^* r_2 \end{bmatrix}
=
\partial f_1^* r_1 + \partial f_2^* r_2
```
It's a **sum**! 🎉 So we initialize ``r_0 = 0`` and each function using ``x`` accumulates its contribution with ``r_0 \mathrel{\raise{0.19ex}{\scriptstyle+}} = \partial f_i^* r_i``.
""")
```

Forward tangents



Reverse tangents



Error message

```
InterruptException:
```

```
qa(md"Why is ``\partial\text{dup}^*`` a sum ?", md"The Jacobian is ``\partial\text{dup} = \begin{bmatrix} 1\\1\\1\end{bmatrix}``. In reverse mode, we multiply by the adjoint (why ? See next lecture!) of the Jacobian (here the transpose) ``\partial\text{dup}^* = \begin{bmatrix}
```

```
1 & 1 & 1\end{bmatrix}``. Left-multiplying a vector with a row vector of ones results in its sum.")
```

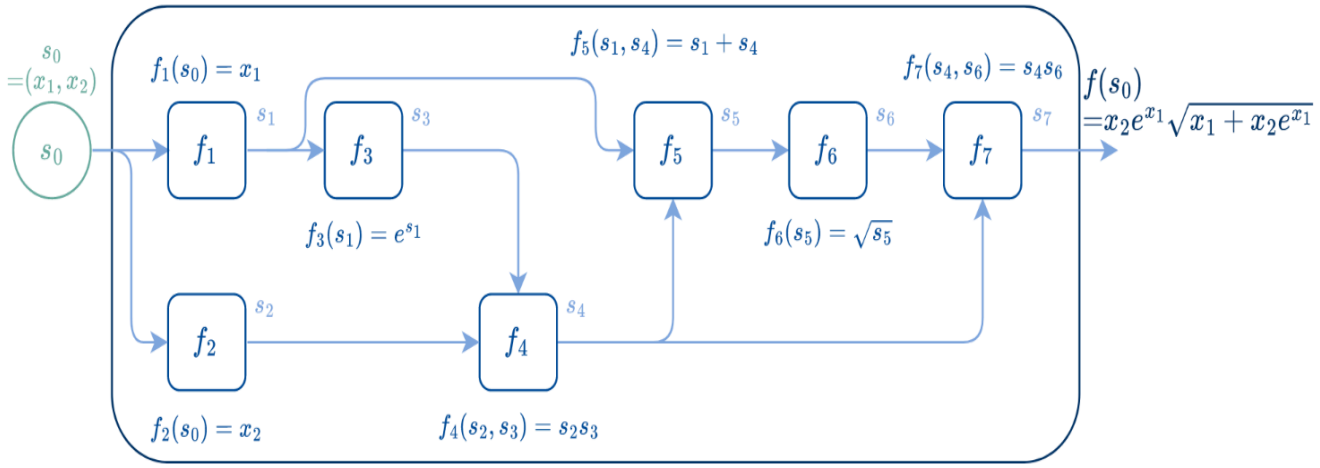
Reading hidden code

Computation graph

We build the graph of

$$f(x_1, x_2) = x_2 e^{x_1} \sqrt{x_1 + x_2 e^{x_2}}$$

at $x_1 = 1, x_2 = 2$. The slider first reveals the forward evaluation of each node v_i, then accumulates the adjoints $\bar{v}_i = \partial f / \partial v_i$ backwards, one node at a time.



Error message

```
InterruptException:
```

```
1 qa(md"Can this directed graph have cycles ?", md"No, it is a Directed Acyclic Graph. (DAG)")
```

Reading hidden code

Error message

```
InterruptException:
```

```
qa(md"What happens if ``f_4`` is handled before ``f_5`` in the backward pass ?",
md"""
``\partial f / \partial s_4`` is the sum of its contribution from ``f_5`` and
``f_7``. We must wait for ``f_5`` to be handled before we turn to ``f_4``.
""")
```

Reading hidden code

Error message

```
InterruptException:
```

```
qa(md"How to prevent this from happening ?", md"We should first compute a
[*topological ordering*](https://en.wikipedia.org/wiki/Topological_sorting) and
then follow this order.")
```

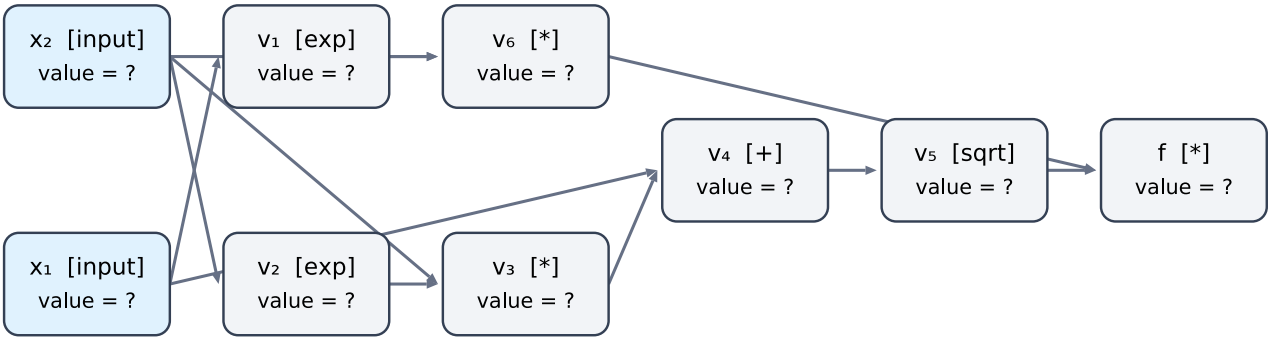
Reading hidden code

Reverse mode in action

$$f(x_1, x_2) = x_2 e^{x_1} \sqrt{x_1 + x_2 e^{x_2}}$$

◀ ▶ 1

Computation graph



Comparison

- Forward mode of $f(x)$ with dual numbers `Dual.(x, v)` computes Jacobian-Vector Product (JVP) $J_f(x) \cdot v$
- Reverse mode of $f(x)$ computes Vector-Jacobian Product (VJP) $v^\top J_f(x)$ or in other words $J_v(x)^\top v$

Error message

```
InterruptException:
```

```
qa(md"How can we compute the full Jacobian ?", md"By computing a JVP (resp. VJP) with a one-hot vector, we get a column (resp. row) of the jacobian.")
```

👀 *Reading hidden code*

Error message

```
InterruptException:
```

```
qa(md"When is each mode faster than the other one to compute the full Jacobian ?", md"If ``f: \mathbb{R}^n \to \mathbb{R}^m``, then computing the full Jacobian requires ``n`` JVP or ``m`` VJP. So
* if ``n \gg m``, then reverse mode is faster;
* if ``m \gg n``, then forward mode is faster;
* if ``m \approx n``, then it's a close call.")
```

👀 *Reading hidden code*

Error message

```
InterruptException:
```

```
qa(md"When is the speed of numerical differentation comparable to autodiff ?", md"""

```

```
With numerical differentiation, we compute a JVP with ``\partial f / \partial x_i
\approx (f(x_1, \ldots, x_{i - 1}, x_i + \epsilon, x_{i + 1}, x_n) - f(x_1, \ldots,
x_n)) / \epsilon``.
For this JVP, we need to evaluate ``f`` twice. On the other hand, forward mode
evaluates ``f`` once but with dual numbers as inputs so this evaluation is probably
around twice as expensive as evaluating ``f`` with `Float64` numbers. So the cost
of a JVP should be roughly the same for numerical differentiation and forward
differentiation.

Numerical differentiation may however need to increase its number of evaluations in
order to improve its accuracy while forward is accurate (up to floating point
rounding errors).
```

👀 *Reading hidden code*

From forward to reverse mode

Consider $f(x_1, x_2) = x_1^2 + x_2^2$ at $s_0 = (1, 2)$. We decompose it as $f = f_2 \circ f_1$ and compute the **local** Jacobians:

$$
\begin{aligned}
f_1(x_1, x_2) &= \begin{bmatrix} x_1^2 \\ x_2^2 \end{bmatrix} & s_1 = f_1(s_0) &= \begin{bmatrix} 1 \\ 4 \end{bmatrix} & \partial f_1(s_0) &= \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix} \\
f_2(x_1, x_2) &= x_1 + x_2 & s_2 = f_2(s_1) &= 5 & \partial f_2(s_1) &= \begin{bmatrix} 1 & 1 \end{bmatrix}
\end{aligned}
$$

Forward mode computes one column of the Jacobian $\partial f(s_0)$ for each one-hot tangent t_0:

$$
\begin{aligned}
t_0 &= \begin{bmatrix} 1 \\ 0 \end{bmatrix} & t_1 = \partial f_1(s_0) \cdot t_0 &= \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix} & t_2 = \partial f_2(s_1) \cdot t_1 &= \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \end{bmatrix} = 2 \\
t_0 &= \begin{bmatrix} 0 \\ 1 \end{bmatrix} & t_1 = \partial f_1(s_0) \cdot t_0 &= \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 4 \end{bmatrix} & t_2 = \partial f_2(s_1) \cdot t_1 &= \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 4 \end{bmatrix} = 4
\end{aligned}
$$

From forward to reverse mode

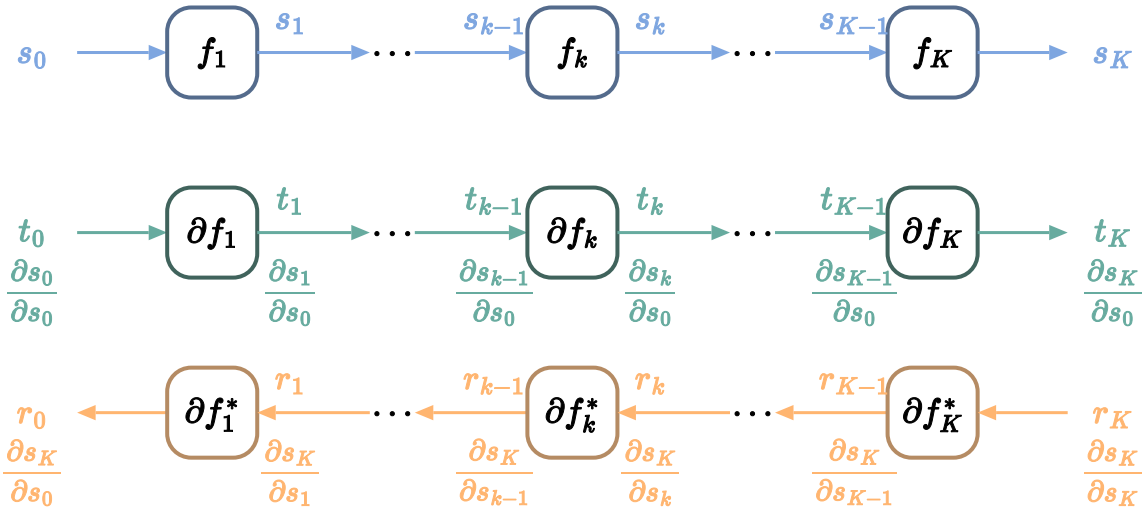
Concatenating the two columns gives the full Jacobian. We can then factor $\partial f_2(s_1)$ and then $\partial f_1(s_0)$ out of the concatenation:

$$
\begin{aligned}
\partial f(s_0) &= \begin{bmatrix} 2 & 4 \end{bmatrix} \\
&= \begin{bmatrix} \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} & \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \end{bmatrix} \\
&= \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} & \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \end{bmatrix} \\
&= \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix} \begin{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} & \begin{bmatrix} 0 \\ 1 \end{bmatrix} \end{bmatrix} \\
&= \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix} \cancel{\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}}
\end{aligned}
$$

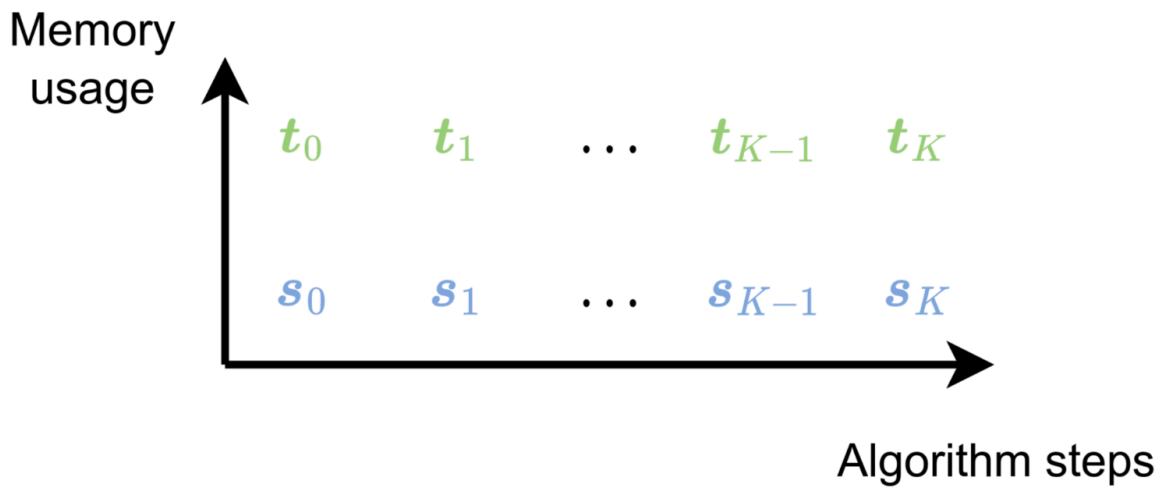
The concatenation of the one-hot tangents is the identity so it disappears! We are left with $\partial f_2(s_1) \cdot \partial f_1(s_0)$ that we can compute from left to right in a single pass: this is reverse mode!

$$
r_2 = 1 \qquad r_1 = r_2 \cdot \partial f_2(s_1) = \begin{bmatrix} 1 & 1 \end{bmatrix} \qquad r_0 = r_1 \cdot \partial f_1(s_0) = \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix} = \begin{bmatrix} 2 & 4 \end{bmatrix}
$$

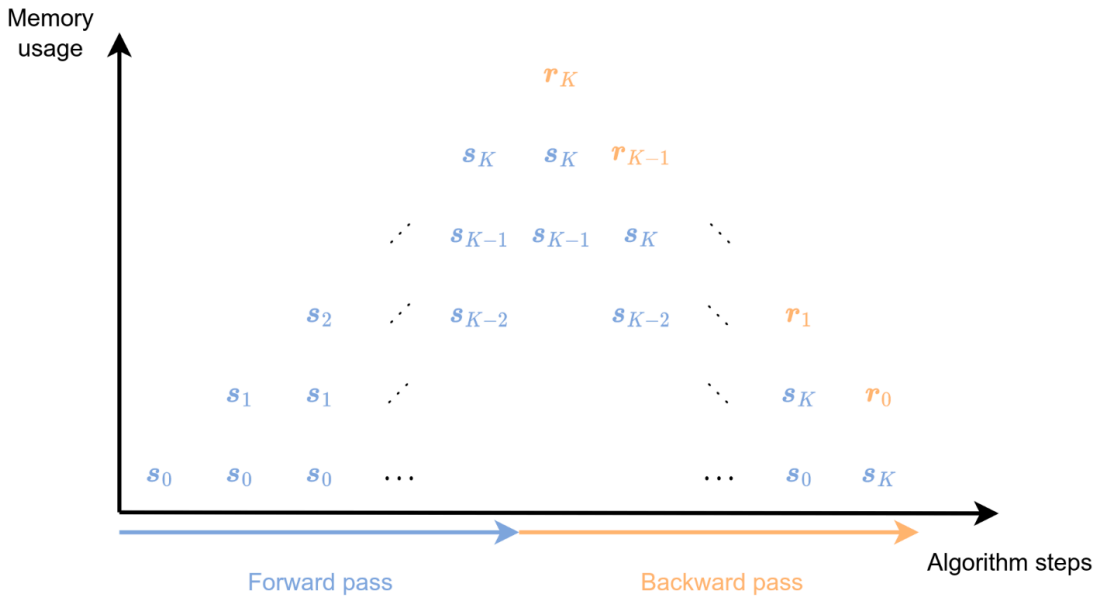
Interpretation of tangents



Memory usage of forward mode



Memory usage of reverse mode



Error message

```
InterruptException:
```

```
qa(md"Why don't we show the local Jacobian ``\partial J_k`` in the memory usage ?", md"For a scalar AD, it is best to compute the local Jacobian ``\partial J_k`` and ``s_k`` from ``s_{k-1}`` during the forward pass at the same time because part of the computation for one can be reused for the other one. For instance, if ``f_k(x) = \sin(x)``, it is more efficient to compute ``\sin(x)`` and ``\cos(x)`` at the same time with the Julia function `sincos`. However, as we will see, for a vectorized AD, we usually don't compute a local Jacobian and just use ``s_{k-1}`` to do the backward propagation of ``r_k`` towards ``r_{k-1}``. This is why this picture, which is taken from the book which focuses on vectorized AD doesn't show a memory usage for the local Jacobian.")
```

👀 *Reading hidden code*

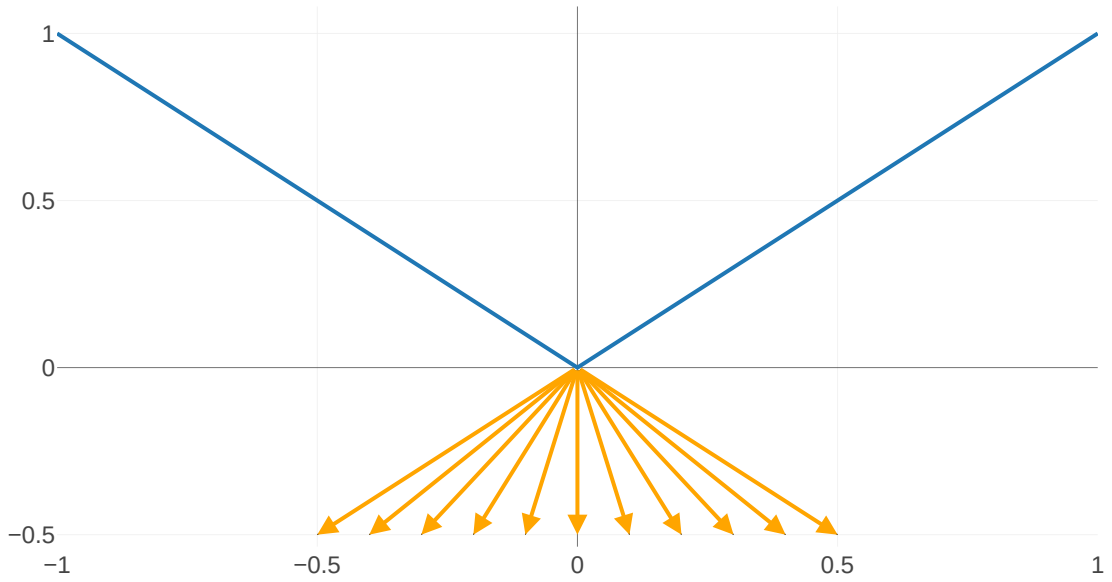
Discontinuity

Error message

```
InterruptException:
```

```
1 qa(md"Is the function ``|x|`` is differentiable at ``x = 0`` ?.", md"No, if we
  approach from the left (that is, ``x < 0``, the function is ``-x``), then the
  derivative is ``-1``.
2 If we approach from the right (that is, ``x > 0``, the function is ``x``), then the
  derivative is ``1``.
3 There is no valid gradient!")
```

Reading hidden code



Error message

```
InterruptException:
```

```
qa(md"What about returning a convex combination of the derivative from the left and right ?", md"Any number between ``-1`` and ``1`` is a valid **subgradient**!
Whereas the gradient is the normal to the **unique** tangent, the subgradient is an element of the **tangent cone**, depicted below. For convex functions, the notion of subgradient appropriately generalizes the notion of gradient for nonsmooth functions.

Note that the notion of subgradient is not defined for nonconvex functions. So instead we use [Clarke differential](https://en.wikipedia.org/wiki/Clarke_generalized_derivative) but this is out of the scope of this course.")
```

Reading hidden code

Forward mode

```
abs (generic function with 1 method)
```

```
abs(x) = ifelse(x < 0, -x, x)
```

```
abs_bis (generic function with 1 method)
```

```
abs_bis(x) = ifelse(x > 0, x, -x)
```

```
Base.isless(x::Dual, y::Real) = isless(x.value, y)
```

```
Base.isless(x::Real, y::Dual) = isless(x, y.value)
```

```
Dual(0, 1)
```

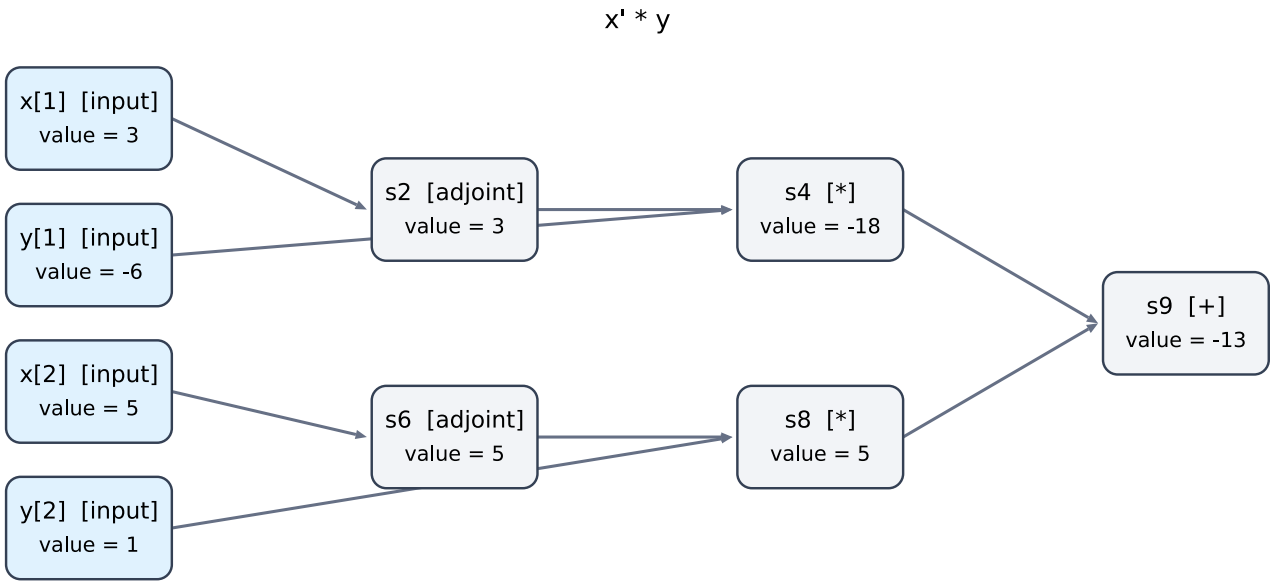
```
abs(Dual(0, 1))
```

```
Dual(0, -1)
```

```
abs_bis(Dual(0, 1))
```

Issues with scalar AD

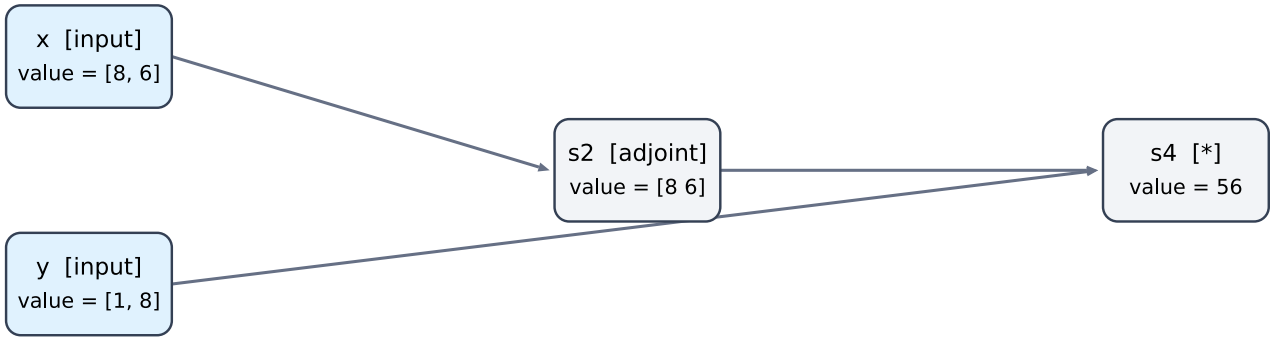
n = 2



Nodes with vector value

n = 2

x' * y



Disadvantages of scalar-valued nodes

- If AD in an interpreted language like Python, the bottleneck of scalar-valued AD becomes the Python interpreter. Solved by using compiled language like Julia/Rust/C/C++/...
- If each node is heap allocated (e.g. with `malloc` in C), not cache friendly. Solved by a centralized storage for all the values of all nodes, e.g. `MathOptInterface.Nonlinear.ReverseAD`'s values vector (TODO hyperlink)
- Not using optimized BLAS routines or GPU kernels for matrix-matrix multiplications. Not solveable for scalar-valued AD, need vector-valued AD!

Neural network

1. Start with an input vector $\boldsymbol{x}$.
2. Apply a matrix product $\boldsymbol{y} = \boldsymbol{W_1 x}$ (or $\boldsymbol{y} = \boldsymbol{W_1 x} + \boldsymbol{b_1}$ with bias).
3. Apply an activation function, say `tanh`, independently to each entry: $\boldsymbol{z_i} = \mathbf{tanh}(\boldsymbol{y_i})$. Using the broadcasting syntax of Julia, we can write $\boldsymbol{z} = \mathbf{tanh.}\,(\boldsymbol{y})$. Usually denoted $\boldsymbol{\sigma(y)}$.
4. Repeat with $\boldsymbol{W_2}$ for next layer etc...

$$\hat{\boldsymbol{y}} = \boldsymbol{W_\ell}\,\mathbf{tanh.}\,(\boldsymbol{W_{\ell-1}}\,\mathbf{tanh.}\,(\cdots \boldsymbol{W_2}\,\mathbf{tanh.}\,(\boldsymbol{W_1 x})))$$

Batching

Applying a neural network to 2 inputs can be done simultaneously so that **matrix-vector** products become **matrix-matrix** product, which better exploit the parallelization capabilities of CPUs and GPUs. Let $\boldsymbol{X} = \begin{bmatrix}\boldsymbol{x^{(1)}} & \boldsymbol{x^{(2)}}\end{bmatrix}$:

$$\begin{aligned}\begin{bmatrix}\sigma(\boldsymbol{W_1 x^{(1)}}) & \sigma(\boldsymbol{W_1 x^{(2)}})\end{bmatrix} &= \sigma(\begin{bmatrix}\boldsymbol{W_1 x^{(1)}} & \boldsymbol{W_1 x^{(2)}}\end{bmatrix}) \\ &= \sigma(\boldsymbol{W_1}\begin{bmatrix}\boldsymbol{x^{(1)}} & \boldsymbol{x^{(2)}}\end{bmatrix}) \\ &= \sigma(\boldsymbol{W_1 X})\end{aligned}$$

With bias, let $\mathbf{1}^\top = \begin{bmatrix}1 & 1\end{bmatrix}$:

$$\begin{bmatrix}\boldsymbol{W_1 x^{(1)}} + \boldsymbol{b_1} & \boldsymbol{W_1 x^{(2)}} + \boldsymbol{b_1}\end{bmatrix} = \begin{bmatrix}\boldsymbol{W_1 x^{(1)}} & \boldsymbol{W_1 x^{(2)}}\end{bmatrix} + \begin{bmatrix}\boldsymbol{b_1} & \boldsymbol{b_1}\end{bmatrix} = \boldsymbol{W_1 X} + \boldsymbol{b_1}\mathbf{1}^\top$$

Two equivalent approaches

$\boldsymbol{b_k}$ is a **column** vector, $\boldsymbol{S_i}, \boldsymbol{X}, \boldsymbol{W_i}, \boldsymbol{Y}$ are matrices.

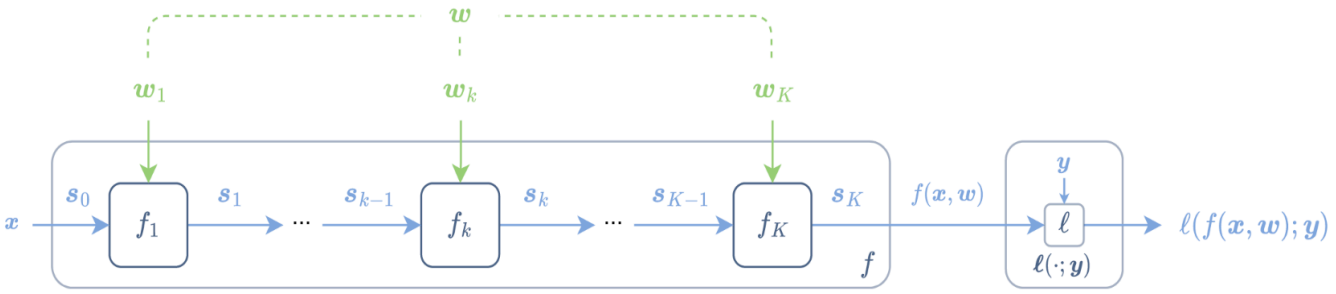
Right-to-left

$$
\begin{aligned}
S_0 &= X \\
S_{2k-1} &= W_k S_{2k-2} + b_k \mathbf{1}^\top \\
S_{2k} &= \sigma(S_{2k-1}) \\
S_{2H+1} &= W_{k+1} S_{2H} \\
S_{2H+2} &= \ell(S_{2H+1}; Y)
\end{aligned}
$$

Left-to-right

$$
\begin{aligned}
S_0 &= X \\
S_{2k-1} &= S_{2k-2} W_k + \mathbf{1} b_k^\top \\
S_{2k} &= \sigma(S_{2k-1}) \\
S_{2H+1} &= S_{2H} W_{k+1} \\
S_{2H+2} &= \ell(S_{2H+1}; Y)
\end{aligned}
$$

Evaluation



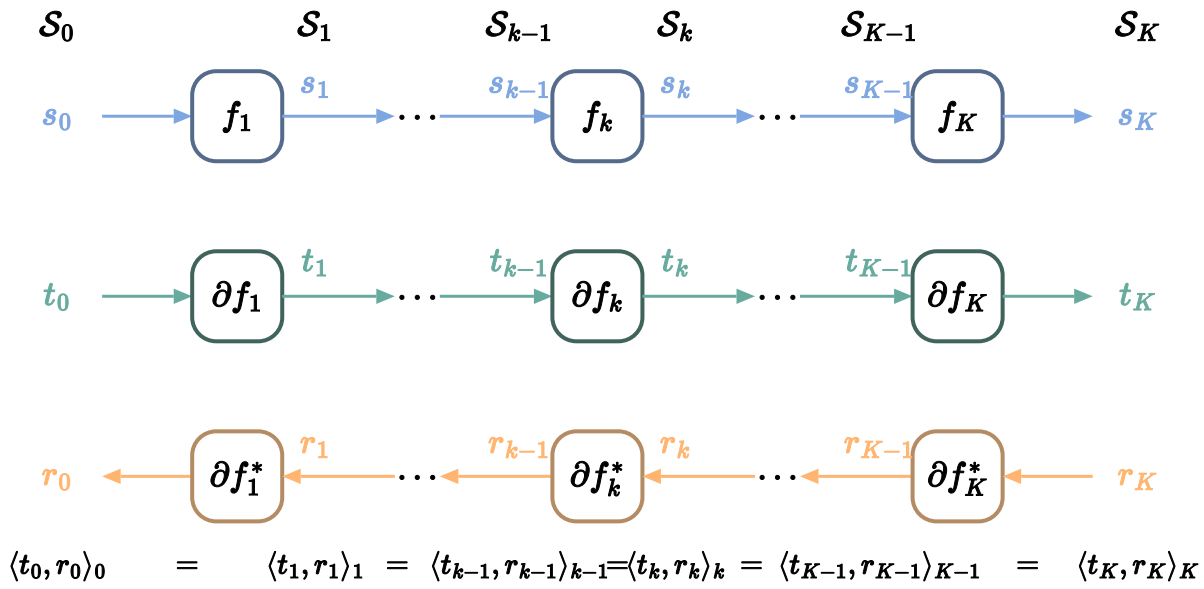
Spaces and scalar products

Let $(\mathcal{S}_k)_k$ be the spaces for which $s_k, t_k, r_k \in \mathcal{S}_k$ and $f_k : \mathcal{S}_{k-1} \to \mathcal{S}_k$.

In the vectorized case, the Jacobian ∂f_k may be expressed more naturally as a linear map $\partial f_k : \mathcal{S}_{k-1} \to \mathcal{S}_k$ and $t_k = \partial f_k(t_{k-1})$. To obtain the matrix-form of the Jacobian, we need to fix a basis for $\mathcal{S}_{k-1}$ and $\mathcal{S}_k$ and vectorize t_k and t_{k-1} according to that basis which is often cumbersome.

Instead, we choose a scalar products $\langle \cdot, \cdot \rangle_k$ for $\mathcal{S}_k$ and the **adjoint** of the Jacobians according to these bases in the backward pass. For instance, if $\mathcal{S}_K = \mathcal{R}$, $t_K = \langle t_K, 1 \rangle_K$ for the canonical scalar product of $\mathcal{R}$ so we pick $r_K = 1$. We then search for a way to obtain $r_0(r_K)$ so that $\langle t_0, r_0(r_K) \rangle_0 = \langle \partial f_K(\cdots \partial f_1(t_0), r_K \rangle_K$. Then, for instance if $\mathcal{S}_0 = \mathbb{R}^n$, with the canonical scalar product $\langle x, y \rangle_0 = x^\top y$, we have $\partial_i f(s_0) = \langle e_i, r_0(r_K) \rangle_0 = (r_0(r_K))_i$.

Spaces and scalar products



Adjoint

The *adjoint* of a linear map $A : \mathcal{S}_1 \to \mathcal{S}_2$ for scalar products $\langle \cdot, \cdot \rangle_1$ for $\mathcal{S}_1$ and $\langle \cdot, \cdot \rangle_2$ for $\mathcal{S}_2$ is the linear map A^* such that

$$\forall x \in \mathcal{S}_1, y \in \mathcal{S}_2, \qquad \langle A(x), y \rangle_2 = \langle x, A^*(y) \rangle_1.$$

For our Jacobians:

$$\begin{aligned} \langle t_k, r_k \rangle_k &= \langle \partial f_k(t_{k-1}), r_k \rangle_k \\ &= \langle t_{k-1}, \partial f_k^*(r_k) \rangle_{k-1} \end{aligned}$$

so by defining the backward pass with $r_{k-1} = \partial f_k^*(r_k)$, we get

$$\langle t_k, r_k \rangle = \langle t_{k-1}, r_{k-1} \rangle$$

By induction, we get

$$\langle t_0, r_0 \rangle = \langle t_K, r_K \rangle$$

Error message

```
InterruptException:
```

```
qa(md"We are free to choose any scalar product ``\langle \cdot, \cdot \rangle_k``. Does this choice affect the Jacobian values as well as gradient of ``f`` ?",
   md"""
If we change the scalar product ``\langle \cdot, \cdot \rangle_k``, it changes both the adjoint ``\partial f^*_k`` and ``\partial f^*_{k+1}``. But it does not affect the gradient of ``f`` as the changes made to both these Jacobian cancel out. The composition ``(\partial f^*_k) \circ (\partial f^*_{k+1})`` is unaffected by the choice of the scalar product ``\langle \cdot, \cdot \rangle_k``.

For example, suppose ``\mathcal{S}_0 = \mathcal{S}_1 = \mathcal{S}_2 = \mathbb{R}`` and ``f_1(x) = x^2`` and ``f_2(x) = \exp(x)``. For ``s_0 = 1``, we have ``s_1 = 1``, ``s_2  = e``, ``\partial f_1(1) = 2``, ``\partial f_2(1) = e``.
So forward math computes ``(f_2 \circ f_1)'(1) = \partial f_1(1) \cdot \partial f_2(1) = 2e``.

For ``\langle x, y \rangle_0 = \rangle x, y \rangle_1 = \rangle x, y \rangle_2 = xy``, we have ``\partial f_1^*(1) = 2``, ``\partial f_2^*(1) = e``. So the reverse mode computes ````(f_2 \circ f_1)'(1) = \partial f_1^*(1) \cdot \partial f_2^*(1) = 2e``.

For ``\rangle x, y \rangle_1 = 2xy``, we have ``\partial f_1^*(1) = 1``, ``\partial f_2^*(1) = 2e``. So the reverse mode computes ````(f_2 \circ f_1)'(1) = \partial f_1^*(1) \cdot \partial f_2^*(1) = 2e``. As we can see, by scaling the scalar product by 2, one jacobian gets scaled by 2 and one by 1/2. When we multiply them together, they cancel out.
""")
```

Reading hidden code

Matrix multiplication (Vectorized way)

Kronecker product properties:

$$\mathbf{vec}(AXB) = (B^\top \otimes A)\mathbf{vec}(X) \qquad (A \otimes B)^\top = A^\top \otimes B^\top$$

Deriving VJP and JVP for matrix product:

$$
\begin{aligned}
F(X) &= AX \\
G(\mathrm{vec}(X)) \triangleq \mathrm{vec}(F(X)) &= (I \otimes A)\mathrm{vec}(X) \\
J_G &= (I \otimes A) \\
J_G^\top \mathrm{vec}(R) &= (I \otimes A^\top)\mathrm{vec}(R) \\
\partial F^*[R] = \mathrm{mat}(J_G^\top \mathrm{vec}(R)) &= A^\top R
\end{aligned}
$$

Error message

```
InterruptException:
```

```
qa(md"How should we store the Jacobian in the forward pass to save it for the backward pass ?",
md"The matrix ``I \otimes A`` is block diagonal with the same block repeated so the structure is crucial to exploit. Storing ``A`` is enough.")
```

👀 *Reading hidden code*

Matrix multiplication (Scalar product way)

For the scalar product

$$\langle X, Y\rangle = \sum_{i,j} X_{ij}Y_{ij} = \sum_{i,j}(X \odot Y)_{ij} = \langle \mathrm{vec}(X), \mathrm{vec}(Y)\rangle = \mathrm{tr}(XY^\top), \quad A^* = A^\top$$

Now, given a forward tangent T and a reverse tangent R

$$\langle AT, R\rangle = \langle T, A^\top R\rangle$$

so the backward pass computes $A^\top R$.

Error message

```
InterruptException:
```

```
qa(md"How to prove that ``A^* = A^\top`` ?", md"""
We have ``\langle X, Y \rangle = \text{tr}(XY^\top)`` ([why ?](https://en.wikipedia.org/wiki/Trace_(linear_algebra)#Trace_of_a_product)) so,
```

```
using the [cyclic property of the trace]
(https://en.wikipedia.org/wiki/Trace_(linear_algebra)#Cyclic_property)
```math
\langle AX, Y \rangle = \text{tr}(AXY^\top) = \text{tr}(XY^\top A) = \text{tr}
(X(A^\top Y)^\top) = \langle X, A^\top Y \rangle.
```
"""
```

👀 *Reading hidden code*

Inner products

$\langle\rangle_i$, the last one should be just product of scalars to use $r_K = 1$, the first one should be canonical to be one-hot. The inner ones can be anything. Multiplying it by 2 makes one adjoint be multiplied by 2, the other by 1/2, it just cancels out.

Broadcasting (Vectorized way)

Consider applying a scalar function f (e.g. $\tanh$ to each entry of a matrix X.)

$$\begin{aligned}
(F(X))_{ij} &= f(X_{ij}) = f.(X) \\
G(\text{vec}(X)) \triangleq \text{vec}(F(X)) &= \text{vec}(f.(X)) \\
J_G &= \text{Diag}(\text{vec}(f'.(X))) \\
J_G \text{vec}(T) &= \text{Diag}(\text{vec}(f'.(X)))\text{vec}(T) \\
\partial F[T] = \text{mat}(J_G \text{vec}(T)) &= f'.(X) \odot T \\
J_G^\top \text{vec}(R) &= \text{Diag}(\text{vec}(f'.(X)))\text{vec}(R) \\
\partial F^*[R] = \text{mat}(J_G^\top \text{vec}(R)) &= f'.(X) \odot R
\end{aligned}$$

Broadcasting (Scalar product way)

$$\langle f'.(X) \odot T, R\rangle = \langle T, f'.(X) \odot R\rangle.$$

Error message

```
InterruptException:
```

```
qa(md"Let ``A(X) = B \odot X``, what is the adjoint ``A^*`` ?", md"""
```math
\begin{align}
   \langle B \odot X, Y \rangle
   & =
   \sum_{ij} ((B \odot X) \odot Y)_{ij}\\
   & =
   \sum_{ij} (X \odot (B \odot Y))_{ij}\\
   & =
   \langle X, B \odot Y \rangle.
\end{align}
```
So ``A`` is self-adjoint, i.e., ``A^* = A``.
""")
```

👀 *Reading hidden code*

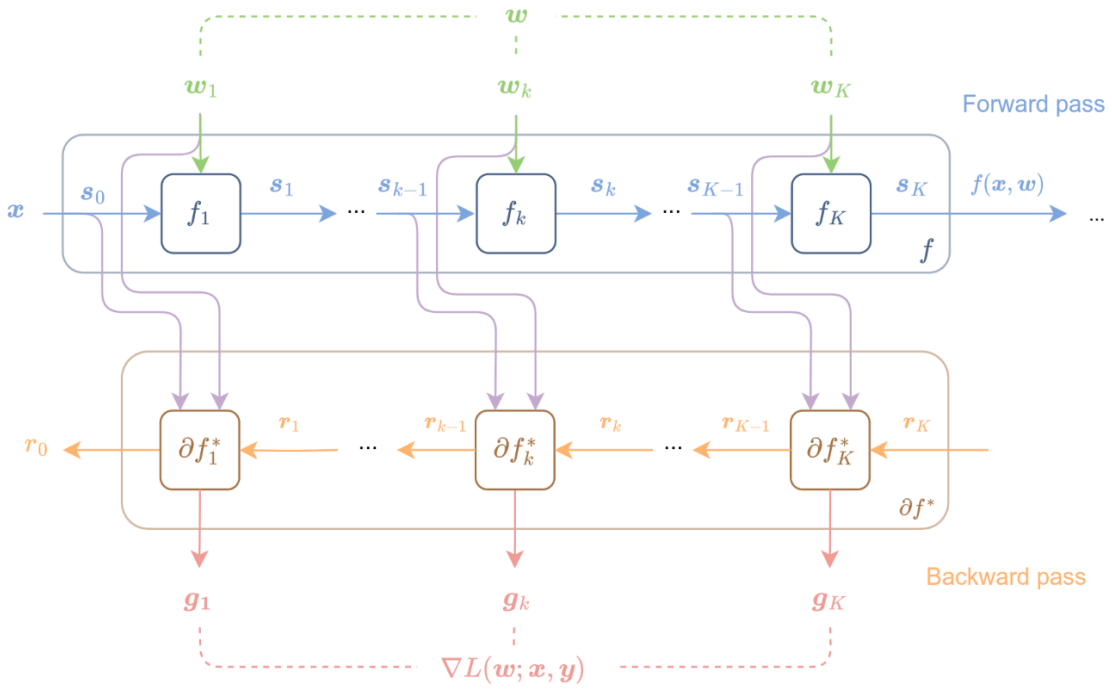
Error message

> **InterruptException:**

```
qa(md"What should be saved for the backward pass ?", md"The Jacobian is diagonal, we just need to compute and store the values on the diagonal: ``f.(X)`` or we can also just store ``X`` and compute ``f.(X)`` during the backward pass.")
```

👀 *Reading hidden code*

Putting everything together



Product of Jacobians

Suppose that we need to differentiate a composition of functions: $(f_n \circ f_{n-1} \circ \cdots \circ f_2 \circ f_1)(w)$. For each function, we can compute a jacobian given the value of its input. So, during a forward pass, we can compute all jacobians. We now just need to take the product of these jacobians:

$$J_n J_{n-1} \cdots J_2 J_1$$

While the product of matrices is associative, its computational complexity depends on the order of the multiplications! Let $d_i \times d_{i-1}$ be the dimension of J_i.

Error message

```
InterruptException:
```

```
qa(md"What is the complexity of forward mode", md"""
If the product is computed from right to left:
```

```
```math
\begin{align}
  J_{1,2} & = J_2 J_1 && \Omega(d_2d_1d_0)\\
  J_{1,3} & = J_3 J_{1,2} && \Omega(d_3d_2d_0)\\
  J_{1,4} & = J_4 J_{1,3} && \Omega(d_4d_3d_0)\\
  \vdots & \quad \vdots\\
  J_{1,n} & = J_n J_{1,(n-1)} && \Omega(d_nd_{n-1}d_0)\\
\end{align}
```
we have a complexity of
``\Omega(\sum_{i=2}^n d_id_{i-1}d_0)``.
"""
```

👀 *Reading hidden code*

Error message

```
InterruptException:
```

```
qa(md"What is the complexity of reverse mode", md"""
The adjoint trick gives
```math
\langle J_n J_{n-1} \cdots J_2 J_1 \partial w, \partial y \rangle
=
\langle \partial w, J_1^\top J_2^\top \cdots J_{n-1}^\top J_n^\top \partial y
\rangle.
```
So reverse differentation corresponds to multiplying the adjoint from right to left
or equivalently the original matrices from left to right.
This means computing the product in the following order:
```math
\begin{align}
  J_{(n-1),n} & = J_n J_{n-1} && \Omega(d_nd_{n-1}d_{n-2})\\
  J_{(n-2),n} & = J_{(n-1),n} J_{n-2} && \Omega(d_nd_{n-2}d_{n-3})\\
  J_{(n-3),n} & = J_{(n-2),n} J_{n-3} && \Omega(d_nd_{n-3}d_{n-4})\\
  \vdots & \quad \vdots\\
  J_{1,n} & = J_{2,n} J_1 && \Omega(d_nd_1d_0)\\
\end{align}
```
We have a complexity of
```math
\Omega(\sum_{i=1}^{n-1} d_nd_id_{i-1}).
```
""")
```

👀 *Reading hidden code*

Error message

```
InterruptException:
```

```
qa(md"What about the complexity of meeting in the middle between ``k`` and ``k+1``?",
  md"""
We can also write
```math
\langle J_n J_{n-1} \cdots J_2 J_1 \partial w, \partial y \rangle
=
\langle J_k J_{k-1} \cdots J_2 J_1\partial w, J_{k+1}^\top J_{k+2}^\top \cdots J_{n-1}^\top J_n^\top \partial y \rangle.
```
This corresponds to multiplying starting from some ``d_k`` where ``1 < k < n``.
We would then first compute the left side:
```math
\begin{align}
  J_{k+1,k+2} & = J_{k+2} J_{k+1} && \Omega(d_{k+2}d_{k+1}d_{k})\\
  J_{k+1,k+3} & = J_{k+3} J_{k+1,k+2} && \Omega(d_{k+3}d_{k+2}d_{k})\\
  \vdots & \quad \vdots\\
  J_{k+1,n} & = J_{n} J_{k+1,n-1} && \Omega(d_nd_{n-1}d_k)
\end{align}
```
then the right side:
```math
\begin{align}
  J_{k-1,k} & = J_k J_{k-1} && \Omega(d_kd_{k-1}d_{k-2})\\
  J_{k-2,k} & = J_{k-1,k} J_{k-2} && \Omega(d_kd_{k-2}d_{k-3})\\
  \vdots & \quad \vdots\\
  J_{1,k} & = J_{2,k} J_1 && \Omega(d_kd_1d_0)\\
\end{align}
```
and then combine both sides:
```math
J_{1,n} = J_{k+1,n} J_{1,k} \qquad \Omega(d_nd_kd_0)
```
we have a complexity of
```math
\Omega(d_nd_kd_0 + \sum_{i=1}^{k-1} d_kd_id_{i-1} + \sum_{i=k+2}^{n} d_id_{i-1}d_k).
```
""")
```

👀 *Reading hidden code*

Error message

```
InterruptException:
```

```
qa(md"Which mode should be used depending on the ``d_i`` ?", md"""
We see that we should find the minimum ``d_k`` and start from there. If the minimum is attained at ``k = n``, this corresponds mutliplying from left to right, this is reverse differentiation. If the minimum is attained at ``k = 0``, we should multiply from right to left, this is forward mode. Otherwise, we should start from the middle, this would mean mixing both forward and reverse mode.
""")
```

👀 *Reading hidden code*

Error message

```
InterruptException:
```

```
qa(md"What about neural networks ?", md"""
In that case, ``d_0`` is equal to the number of entries in ``W_1`` added with the number of entries in ``W_2`` while ``d_n`` is ``1`` since the loss is scalar. We should therefore clearly multiply from left to right hence do reverse diff.
""")
```

👀 *Reading hidden code*

Wine example 🍷

```
data = DelimitedFiles.readdlm(
    last(save_image(URL("https://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data"))),
    ',', Float64,
);
```

```
features =
13×178 Matrix{Float64}:
   14.23    13.2     13.16    14.37  …   13.71   13.4    13.27   13.17   14.13
    1.71     1.78     2.36     1.95       5.65    3.91    4.28    2.59    4.1
    2.43     2.14     2.67     2.5        2.45    2.48    2.26    2.37    2.74
   15.6     11.2     18.6     16.8       20.5    23.0    20.0    20.0    24.5
  127.0    100.0    101.0    113.0       95.0   102.0   120.0   120.0    96.0
    2.8      2.65     2.8      3.85  …    1.68    1.8     1.59    1.65    2.05
    3.06     2.76     3.24     3.49       0.61    0.75    0.69    0.68    0.76
    0.28     0.26     0.3      0.24       0.52    0.43    0.43    0.53    0.56
    2.29     1.28     2.81     2.18       1.06    1.41    1.35    1.46    1.35
    5.64     4.38     5.68     7.8        7.7     7.3    10.2     9.3     9.2
    1.04     1.05     1.03     0.86  …    0.64    0.7     0.59    0.6     0.61
    3.92     3.4      3.17     3.45       1.74    1.56    1.56    1.62    1.6
 1065.0   1050.0   1185.0   1480.0      740.0   750.0   835.0   840.0   560.0
```

```
features = permutedims(data[:, 2:end])
```

```
targets =
1×178 Matrix{Float64}:
 1.0  1.0  1.0  1.0  1.0  1.0  1.0  1.0  …  3.0  3.0  3.0  3.0  3.0  3.0  3.0  3.0
```

```
targets = permutedims(data[:, 1:1])
```

```
normalise (generic function with 1 method)
```

```
function normalise(x)
  μ = Statistics.mean(x, dims=2)
  σ = Statistics.std(x, dims=2, mean=μ)
  return (x .- μ) ./ σ
end
```

```
X = 13×178 Matrix{Float32}:
  1.51434    0.245597   0.196325    1.68679   …   0.331822   0.208643    1.39116
 -0.560668  -0.498009   0.0211715  -0.345835      1.73984    0.227053    1.57871
  0.2314    -0.825667   1.10621     0.486554     -0.38826    0.0126963   1.36137
 -1.1663    -2.48384   -0.267982   -0.806975      0.151234   0.151234    1.49872
  1.90852    0.018094   0.0881098   0.9283        1.41841    1.41841    -0.261969
  0.806722   0.567048   0.806722    2.48444   …  -1.12665   -1.03078    -0.391646
  1.03191    0.731565   1.21211     1.4624       -1.3408    -1.35081    -1.27072
 -0.657708  -0.818411  -0.497005   -0.979113      0.547563   1.35108     1.59213
  1.22144   -0.543189   2.12996     1.02925      -0.420888  -0.228701   -0.420888
  0.251009  -0.292496   0.268263    1.18273       2.21798    1.82976     1.78663
  0.361158   0.404908   0.317409   -0.426341  …  -1.60759   -1.56384    -1.52009
  1.84272    1.11032    0.786369    1.18074      -1.48127   -1.39676    -1.42493
  1.01016    0.962526   1.39122     2.32801       0.279786   0.295664   -0.593486
```

```
1 X = Float32.(normalise(features))
```

```
y =
1×178 Matrix{Float32}:
 -1.0  -1.0  -1.0  -1.0  -1.0  -1.0  -1.0  …  1.0  1.0  1.0  1.0  1.0  1.0  1.0  1.0
```

```
1 y = Float32.(targets .- 2)
```

Neural network

h = 16

```
W =
▸ [16×13 Matrix{Float32}:                                                    , 1×16 Ma
   0.44157   0.498274   0.849498   0.561572   …  0.611302   0.556544   0.56834    0.5069
```

```
1 W = [rand(Float32, h, size(X, 1)), rand(Float32, size(y, 1), h)]
```

```
40.77764f0
```

```
1 sum((W[2] * tanh.(W[1] * X) - y).^2) / size(y, 2)
```

Forward mode

```
forward_pass (generic function with 1 method)
```

```
1 function forward_pass(W, X, y)
2     W1, W2 = W
3     y_1 = tanh.(W1 * X)
4     local_der_tanh = 1 .- y_1.^2
5     local_der_mse = 2 * (W2 * y_1 - y) / size(y, 2)
6     return local_der_tanh, local_der_mse
7 end
```

```
forward_diff (generic function with 1 method)
```

```
function forward_diff(W, X, y, j, k)
    W1, W2 = W
    T_1 = onehot(j, axes(W1, 1)) * onehot(k, axes(W1, 2))'
    J_1, J_2 = forward_pass(W, X, y)
    only((W2 * (J_1 .* (T_1 * X))) * J_2') # only: 1x1 matrix -> scalar
end
```

```
forward_diff (generic function with 2 methods)
```

```
function forward_diff(W, X, y)
    [forward_diff(W, X, y, i, j) for i in axes(W[1], 1), j in axes(W[1], 2)]
end
```

Error message

```
Malt.TerminatedWorkerException()
```

```
if h < 200 # Forward Diff start being too slow for `h > 200`
    @time forward_diff(W, X, y)
end
```

Reverse mode

Error message

```
InterruptException:
```

```
qa(md"How to deduce the backward pass for reverse mode from the forward mode ?",
md"""
`W2 * (J_1 .* (T_1 * X))) * J_2'` → The broadcasted `*` is an Hadamard product,
denoted $\odot$. So forward mode is:
```math
W_2 (J_1 \odot (T_1 X)) J_2^\top
```
As this is scalar, it is trivially equal to its scalar product with ``1``. That is,
we start with a reverse tangent ``R = 1``. We can then move everything to the right-
hand side with the adjoint (here, it is the transpose):
```math
```
```

```
\begin{align}
\langle W_2 (J_1 \odot (\partial W_1 X)) J_2^\top, 1 \rangle
& =
\langle J_1 \odot (\partial W_1 X), W_2^\top J_2 \rangle\\
& =
\langle \partial W_1 X, J_1 \odot (W_2^\top J_2) \rangle\\
& =
\langle \partial W_1, (J_1 \odot (W_2^\top J_2))X^\top \rangle\\
\end{align}
```
Now, for the derivative with respect to the entry $W_{i,j}$, we use $\partial W_1 = e_ie_j^\top$ and
```math
\langle e_ie_j^\top, (J_1 \odot (W_2^\top J_2))X^\top \rangle
=
((J_1 \odot (W_2^\top J_2))X^\top)_{ij}
```
So the gradient with respect to W_1 is exactly the matrix $(J_1 \odot (W_2^\top J_2))X^\top$!
"""

👀 *Reading hidden code*

```
reverse_diff (generic function with 1 method)
```

```
function reverse_diff(W, X, y)
    J_1, J_2 = forward_pass(W, X, y)
    (J_1 .* (W[2]' * J_2)) * X'
end
```

Error message

InterruptException:

```
@time reverse_diff(W, X, y)
```

🚀 GPU acceleration ⚡

Error message

```
InterruptException:
```

```
if CUDA.functional()
    X_gpu = CUDA.CuArray(X)
    y_gpu = CUDA.CuArray(y)
    W_gpu = CUDA.CuArray.(W)
    @time reverse_diff(W_gpu, X_gpu, y_gpu)
end
```

Error message

```
InterruptException:
```

```
qa(md"Why is the GPU version slower than the CPU version ?",
md"""
The fixed cost of launching the GPU kernels is larger than the gain obtained by the GPU acceleration. It is only worth it for larger `h`.
`h` = $(h_slider).
""")
```

👀 *Reading hidden code*

Second-order

Consider a function $f : \mathbb{R}^n \to \mathbb{R}$, we want to compute the Hessian $\nabla^2 f(x)$, defined by

$$(\nabla^2 f(x))_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

Application: Given the optimization problem:

$$\begin{aligned} &\min_x f(x) \\ &\quad g_i(x) = 0 \quad \forall i \in \{1, \dots, m\} \end{aligned}$$

The Hessian of the Lagrangian $\mathcal{L}(x, \lambda) = f(x) - \lambda_1 g_1(x) - \cdots - \lambda_m g_m(x)$ is obtained as

$$\nabla^2_x \mathcal{L}(x, \lambda) = \nabla^2 f(x) - \sum_{i=1}^{m} \lambda_i \nabla^2 g_i(x)$$

Second-order AD

Error message

```
InterruptException:
```

```
qa(md"How can the Hessian of ``f`` be computed given an AD for Jacobian and gradient.", md"""
Define the function ``g(x) = \nabla f(x)``, the Hessian of ``f`` is then the Jacobian of ``g``:
``\nabla^2 f(x) = J_g(x)``. See the solutions of `LabAD`!
""")
```

👀 *Reading hidden code*

Error message

```
InterruptException:
```

```
qa(md"Does the AD need to be the same for the gradient and the Jacobian ?",
md"""
No. Given an implementation of reverse mode and forward mode, this gives 4
possibilities depending on which mode is used for the gradient and Jacobian.
""")
```

Reading hidden code

Notation

- Let $f_k : \mathbb{R}^{d_{k-1}} \to \mathbb{R}^{d_k}$. $\partial f_k \triangleq \partial f_k(s_{k-1}) \in \mathbb{R}^{d_k \times d_{k-1}}$, $\partial^2 f_k \triangleq \partial^2 f_k(s_{k-1}) \in \mathbb{R}^{d_k \times d_{k-1} \times d_{k-1}}$ is a 3D array/tensor.
- Given $v \in \mathbb{R}^{d_{k-1}}$, by the product $(\partial^2 f_k \cdot v) \in \mathbb{R}^{d_k \times d_{k-1}}$, we denote the contraction of the 3rd (or 2nd since the tensor is symmetric over its last 2 dimensions) dimension:

$$(\partial^2 f_k \cdot v)_{ij} = \sum_{l=1}^{d_{k-1}} (\partial^2 f_k)_{ijl} \cdot v_l$$

- Given $u \in \mathbb{R}^{d_k}$, by the product $(u \cdot \partial^2 f_k) \in \mathbb{R}^{d_{k-1} \times d_{k-1}}$, we denote the contraction of the 1st dimension.

$$(u \cdot \partial^2 f_k)_{ij} = \sum_{l=1}^{d_k} u_l \cdot (\partial^2 f_k)_{lij}$$

- Both $\partial^2 f_k \cdot v$ and $u \cdot \partial^2 f_k$ are matrices so then we're back to matrix notations.

Chain rule

$$\begin{aligned}
\frac{\partial^2 (f_2 \circ f_1)}{\partial x_i \partial x_j} &= \frac{\partial}{\partial x_j} \left(\frac{\partial (f_2 \circ f_1)}{\partial x_i} \right) \\
&= \frac{\partial}{\partial x_j} \left(\partial f_2 \cdot \frac{\partial f_1}{\partial x_i} \right) \\
&= \left(\partial^2 f_2 \cdot \frac{\partial f_1}{\partial x_j} \right) \cdot \frac{\partial f_1}{\partial x_i} + \partial f_2 \cdot \frac{\partial^2 f_1}{\partial x_i \partial x_j}
\end{aligned}$$

In terms of the matrices $J_k = \partial f_k$ and $H_{kj} = \frac{\partial}{\partial x_j} J_k = \partial^2 f_k \cdot \frac{\partial s_{k-1}}{\partial x_j}$, it becomes

$$\frac{\partial^2 (f_2 \circ f_1)}{\partial x_i \partial x_j} = H_{2j} \cdot \frac{\partial f_1}{\partial x_i} + J_2 \cdot \frac{\partial^2 f_1}{\partial x_i \partial x_j}$$

Forward on forward

Given $\mathbf{Dual}(s_1, t_1)$ with $s_1 = \mathbf{Dual}(f_1(x), \frac{\partial f_1}{\partial x_j})$ and $t_1 = \mathbf{Dual}(\frac{\partial f_1}{\partial x_i}, \frac{\partial^2 f_1}{\partial x_i \partial x_j})$

1. Compute $s_2 = f_2(s_1) = (f_2(f_1(x)), J_2 \cdot \frac{\partial f_1}{\partial x_j}) = ((f_2 \circ f_1)(x), \partial(f_2 \circ f_1)/\partial x_j)$
2. Compute $J_{f_2}(s_1)$ which gives $\mathbf{Dual}(J_2, H_{2j})$
3. Compute

$$\begin{aligned} J_{f_2}(s_1) \cdot t_1 &= \mathbf{Dual}(J_2, H_{2j}) \cdot \mathbf{Dual}(\frac{\partial f_1}{\partial x_i}, \frac{\partial^2 f_1}{\partial x_i \partial x_j}) \\ &= \mathbf{Dual}(J_2 \cdot \frac{\partial f_1}{\partial x_i}, J_2 \cdot \frac{\partial^2 f_1}{\partial x_i \partial x_j} + H_{2j} \cdot \frac{\partial f_1}{\partial x_i}) \\ &= \mathbf{Dual}(\frac{\partial (f_2 \circ f_1)}{\partial x_i}, \frac{\partial^2 (f_2 \circ f_1)}{\partial x_i \partial x_j}) \end{aligned}$$

Error message

```
InterruptException:
```

```
qa(md"What is the closed form expression for ``t_k`` in terms of the matrices ``J_k`` and ``H_{kj}`` ?",
md"""
We can prove by induction that the second part of the dual number ``t_k`` is:
```math
\frac{\partial^2 f_k}{\partial x_i \partial x_j} = J_k \cdots J_2 H_{1j} e_i + J_k \cdots J_3 H_{2j} J_1 e_i + H_{kj} J_{k-1} \cdots J_1 e_i
```
""")
```

👀 Reading hidden code

Forward on reverse

Forward pass: Given $s_1 = \mathbf{Dual}(f_1(x), \frac{\partial f_1}{\partial x_j})$

1. Compute $s_2 = f_2(s_1)$ → same as forward on forward
2. Compute $J_{f_2}(s_1)$ → same as forward on forward

Reverse pass: Given $r_2 = \text{Dual}((r_2)_1, (r_2)_2)$, compute

$$\begin{aligned} r_2 \cdot J_2 &= \text{Dual}((r_2)_1, (r_2)_2) \cdot \text{Dual}(J_2, H_{2j}) \cdot \\ &= \text{Dual}((r_2)_1 \cdot J_2, (r_2)_2 \cdot J_2 + (r_2)_1 \cdot H_{2j}) \end{aligned}$$

Error message

```
InterruptException:
```

```
qa(md"Which value of ``r_k`` is solution for this recurrence equation ?",
md"""
We find ``r_k = \text{Dual}(\frac{\partial f}{\partial s_k}, \frac{\partial^2 f}{\partial s_k \partial x_j})`` as solution:
```math
\begin{align}
(r_1)_2 = (r_2)_2\cdot J_2+(r_2)_1\cdot H_{2j}
& = \frac{\partial^2 f}{\partial s_2 \partial x_j} \cdot J_2 + \frac{\partial f}{\partial s_2} \cdot H_{2j}\\
& = \frac{\partial^2 f}{\partial s_2 \partial x_j}\frac{\partial s_2}{\partial s_1} + \frac{\partial f}{\partial s_2}\frac{\partial J_2}{\partial x_j}\\
& = \frac{\partial^2 f}{\partial s_2 \partial x_j}\frac{\partial s_2}{\partial s_1} + \frac{\partial f}{\partial s_2}\frac{\partial^2 s_2}{\partial s_1\partial x_j}\\
& = \frac{\partial}{\partial x_j}\left(\frac{\partial f}{\cancel{\partial s_2}}\cdot\frac{\cancel{\partial s_2}}{\partial s_1}\right)\\
& = \frac{\partial^2 f}{\partial s_1 \partial x_j}
\end{align}
```
""")
```

👀 *Reading hidden code*

Error message

```
InterruptException:
```

```
qa(md"What is the closed form expression for ``r_k`` in terms of the matrices ``J_k`` and ``H_{kj}`` ?",
md"""
We can prove by induction that the second part of the dual number ``r_k`` is:
```math
\frac{\partial^2 f}{\partial s_k \partial x_j}^\top = (J_K \cdots J_{k+1} H_{kj} + J_K \cdots J_{k+2} H_{(k+1)j} J_k + H_{Kj} J_{K-1} \cdots J_k)^\top
```
""")
```

👀 Reading hidden code

Reverse on forward

Forward pass: Given $s_1 = \text{Dual}(f_1(x), \frac{\partial f_1}{\partial x_i})$

1. Forward mode computes
   $s_2 = f_2(s_1) = (f_2(f_1(x)), J_2 \cdot \frac{\partial f_1}{\partial x_i}) = ((f_2 \circ f_1)(x), \partial(f_2 \circ f_1)/\partial x_i)$
2. The reverse mode computes the local Jacobian of this operation : $\partial s_2 / \partial s_1$. The local Jacobian of $(s_1)_1 \mapsto f_2((s_1)_1)$ is J_2. The local Jacobian of $s_1 \mapsto \partial f_2((s_1)_1)(s_1)_2$ is
   $(\partial^2 f_2((s_1)_1) \cdot (s_1)_2, \partial f_2((s_1)_1)) = (\partial^2 f_2(f_1(x)) \cdot \frac{\partial f_1}{\partial x_i}, \partial f_2(f_1(x)) = (H_{2i}, J_2)$

Reverse pass:

$$(r_1)_1 = (r_2)_1 \cdot J_2 + (r_2)_2 \cdot H_{2i}$$
$$(r_1)_2 = (r_2)_2 \cdot J_2$$

Error message

```
InterruptException:
```

```
qa(md"Which value of ``r_k`` is solution for this recurrence equation ?",
md"""
We find ``r_k = \text{Dual}(\frac{\partial f}{\partial s_k}, \frac{\partial^2 f}{\partial s_k \partial x_{\color{red}j}})`` as solution.
""")
```

👀 Reading hidden code

Reverse on reverse

Forward pass (2nd):

1. Forward pass computes $s_2 = f_2(s_1)$ → Jacobian $\partial s_2 / \partial s_1 = J_2$
2. Local Jacobian $J_2 = \partial f_2(s_1)$ → The Jacobian is the 3D array $\partial J_2 / \partial s_1 = \partial^2 f_2$
3. Backward pass computes $r_1 = r_2 \cdot \partial f_2(s_1)$ → Jacobian of $(s_1, r_2) \mapsto r_2 \cdot \partial f_2(s_1)$ is $(r_2 \cdot \partial^2 f_2(s_1), \partial f_2(s_1)) = (r_2 \cdot \partial^2 f_2, J_2)$. Note that here $r_2 \in \mathbb{R}^{d_k}$ is multiplying the first dimension of the tensor $\partial^2 f_2(s_1) \in \mathbb{R}^{d_k \times d_{k-1} \times d_{k-1}}$ so the result is a symmetric matrix of dimension $\mathbb{R}^{d_{k-1} \times d_{k-1}}$

Reverse pass (2nd): The result is r_0, let $\dot{r}_k$ be the second-order reverse tangent for r_k and $\dot{s}_k$ be the second-order reverse tangent of s_k. We have

$$\dot{r}_2 = J_2 \cdot \dot{r}_1$$
$$\dot{s}_1 = (r_2 \cdot \partial^2 f_2(s_1)) \cdot \dot{r}_1 + \dot{s}_2 \cdot J_2$$

Error message

```
InterruptException:
```

```
qa(md"Which value of ``\dot{s}_k, \dot{r}_k`` is solution for this recurrence equation ?",
md"""
Starting with ``\dot{s}_0 = e_i``, we have
```math
\begin{align}
r_k & = J_K \cdots J_{k+1}\\
\dot{r}_k & = J_k \cdots J_1 e_i\\
(r_k \cdot \partial^2 f_k) \cdot \dot{r}_{k-1}
& =
r_k \cdot (\partial^2 f_k \cdot \dot{r}_{k-1})\\
& =
r_k \cdot H_{ki}\\
\dot{s}_k & = \sum_{k=1}^K r_k H_{ki} J_{k-1} \cdots J_1
\end{align}
```
So we find ``\dot{s}_k = \frac{\partial^2 f}{\partial s_k \partial x_i}, \dot{r}_k = \frac{\partial f}{\partial s_k}`` as solution.
""")
```

👀 Reading hidden code

Error message

```
InterruptException:
```

```
qa(md"What is the difference with reverse on forward and forward on reverse ?",
md"""
Reverse on reverse computes the product in the order
```math
((J_K \cdots J_{k-1}) \cdot \partial^2 f_k) \cdot (J_k \cdots J_1 e_i)
```
while reverse on forward and forward on reverse compute it in the order
```math
(J_K \cdots J_{k-1}) \cdot (\partial^2 f_k \cdot (J_k \cdots J_1 e_i))
```
""")
```

👀 *Reading hidden code*

Acknowledgements and further readings

- `Dual` is inspired from ForwardDiff
- `Node` is inspired from micrograd
- Here is a good intro to AD
- Figures are from the The Elements of Differentiable Programming book

The End

Utils

```
using PlutoUI, PlutoUI.ExperimentalLayout, HypertextLiteral, PlutoTeachingTools, Statistics, CUDA, OneHotArrays
```

```
import PlotlyLight
```

```
import ComputationGraphExplorer as CGE
```

```
_latex (generic function with 1 method)
```

```
begin
    using LaTeXStrings
    import MathTeXEngine
    # Luxor asks Cairo for MathTeXEngine's fonts by name so we register them
    # to avoid having to install them on the system
    let fontconfig = Base.require(Base.PkgId(Base.UUID("a3f928ae-7b40-5064-980b-68af3947d34b"), "Fontconfig_jll"))
        add_font = Base.Libc.Libdl.dlsym(fontconfig.libfontconfig_handle, :FcConfigAppFontAddFile)
        dir = joinpath(pkgdir(MathTeXEngine), "assets", "fonts", "NewComputerModern")
        for file in readdir(dir; join = true)
            if endswith(file, ".otf")
                ccall(add_font, Cint, (Ptr{Cvoid}, Cstring), C_NULL, file)
            end
        end
    end
    # Luxor draws the italic glyphs of letters with the upright font so we
    # replace them with their Unicode mathematical italic counterpart.
    # The arguments of font commands such as `\mathrm{...}` are left untouched
    # and `\mathcal{X}` is replaced by the Unicode script letter since
    # MathTeXEngine would use the bold script one.
    _math_italic(c) = c == 'h' ? 'ℎ' : islowercase(c) ? Char(0x1D44E + (c - 'a')) : Char(0x1D434 + (c - 'A'))
    _math_script(c) = get(Dict('B' => 'ℬ', 'E' => 'ℰ', 'F' => 'ℱ', 'H' => 'ℋ', 'I' => 'ℐ', 'L' => 'ℒ', 'M' => 'ℳ', 'R' => 'ℛ'), c, Char(0x1D49C + (c - 'A')))
    function _latex(s)
        s = replace(s, r"\\mathcal\{([A-Z])\}" => m -> string(_math_script(m[end-1])))
        s = replace(s, r"\\(math[a-z]+|text[a-z]*|operatorname)\{[^}]*\}|\\[a-zA-Z]+|[a-zA-Z]" => m -> length(m) == 1 ? string(_math_italic(m[1])) : m)
        return latexstring(s)
    end
end
```

mathjax_figure

```
mathjax_figure(draw; width, height)
```

Luxor only draws the shapes, the labels are overlaid in HTML so that they are typeset by the MathJax of Pluto. `draw(label)` draws the shapes with the origin at the center and adds labels with `label(text, x, y; color, fontsize, class)`. Labels of class `equal` are centered between the labels of class `inner-product` once MathJax has set their width.

```
"""
    mathjax_figure(draw; width, height)

Luxor only draws the shapes, the labels are overlaid in HTML so that
they are typeset by the MathJax of Pluto.
`draw(label)` draws the shapes with the origin at the center and adds labels
with `label(text, x, y; color, fontsize, class)`.
Labels of class `equal` are centered between the labels of class
`inner-product` once MathJax has set their width.
"""
function mathjax_figure(draw; width, height)
    Luxor = CGE.Luxor
    labels = []
    function label(text, x, y; color = "black", fontsize = 14, class = "")
        push!(labels, @htl("""<span class="tex $class" style="position: absolute;
        left: $(x + width / 2)px; top: $(y + height / 2)px; transform:
        translate(-50%, -50%); white-space: nowrap; color: $color; font-size:
        $(fontsize)px;">\$$text\$</span>"""))
    end
    Luxor.Drawing(width, height, :svg)
    Luxor.origin()
    Luxor.background("white")
    draw(label)
    Luxor.finish()
    svg = Luxor.svgstring()
    @htl("""<div style="position: relative; width: $(width)px; height:
    $(height)px;">
    <div style="position: absolute; inset: 0;">$(HTML(svg))</div>
    $labels
    <script>
    // Center each `=` between the inner products once MathJax has set their width
    const root = currentScript.parentElement
    const products = [...root.querySelectorAll(".inner-product")]
    const equals = [...root.querySelectorAll(".equal")]
    const place = () => {
        const box = root.getBoundingClientRect()
        const scale = root.offsetWidth / box.width
```

```
        equals.forEach((equal, i) => {
            const left = products[i].getBoundingClientRect()
            const right = products[i + 1].getBoundingClientRect()
            equal.style.left = `${((left.right + right.left) / 2 - box.left) *
            scale}px`
        })
    }
    const observer = new ResizeObserver(place)
    products.forEach(product => observer.observe(product))
    invalidation.then(() => observer.disconnect())
    </script>
    </div>""")
end
```

chain_figure

```
chain_figure(annotate; width = 640, height = 360)
```

Draw the forward, tangent and reverse chains of $f_K \circ \cdots \circ f_1$ with `mathjax_figure`. Additional labels are added with `annotate(label, geometry)` where `label(text, x, y; color, fontsize, class)` adds a label and `geometry` gives the position of the columns `xs` with their index `ks`.

```
"""
    chain_figure(annotate; width = 640, height = 360)

Draw the forward, tangent and reverse chains of ``f_K \\circ \\cdots \\circ f_1``
with [`mathjax_figure`](@ref).
Additional labels are added with `annotate(label, geometry)` where
`label(text, x, y; color, fontsize, class)` adds a label and `geometry`
gives the position of the columns `xs` with their index `ks`.
"""
function chain_figure(annotate; width = 640, height = 360)
    mathjax_figure(; width, height) do _text
        Luxor = CGE.Luxor
        box_width = 45
        dy_up = 3box_width/8
        Δx = 90
        Δy = 90
        colors = (s = "#7ea6e0", t = "#67ab9f", r = "#ffb570")
        function _arrow(start, finish; rev, color)
            if rev
                start, finish = finish, start
            end
            Luxor.sethue(color)
            Luxor.arrow(start, finish; linewidth = 1.5, arrowheadlength = 10)
        end
        function draw_node(text, x, y; fill_color = "white", stroke_color =
        "#f79009", linewidth = 2, width = box_width, height = box_width)
            center = Luxor.Point(x, y)
            Luxor.sethue(fill_color)
            Luxor.box(center, width, height, 12, :fill)
            Luxor.sethue(stroke_color)
            Luxor.setline(linewidth)
            Luxor.box(center, width, height, 12, :stroke)
            _text(text, x, y)
        end
        function draw_line(f, v, y; rev, color = "black", stroke_color)
            dx_dots = box_width/4
            _text("$(v)_0", -3Δx, y; color)
```

```
            _arrow(Luxor.Point(-3Δx + box_width/2, y), Luxor.Point(-2Δx -
            box_width/2, y); rev, color)
            draw_node("$(f)_1", -2Δx, y; stroke_color)
            _arrow(Luxor.Point(-2Δx + box_width/2, y), Luxor.Point(-Δx - dx_dots,
            y); rev, color)
            _text("$(v)_1", -2Δx + box_width/2 + dy_up, y - dy_up; color)
            _text("\\cdots", -Δx, y)
            _arrow(Luxor.Point(-Δx + dx_dots, y), Luxor.Point(-box_width/2, y);
            rev, color)
            _text("$(v)_{k-1}", -box_width/2 - dy_up, y - dy_up; color)
            draw_node("$(f)_k", 0, y; stroke_color)
            _text("$(v)_k", box_width/2 + dy_up, y - dy_up; color)
            _arrow(Luxor.Point(box_width/2, y), Luxor.Point(Δx - dx_dots, y); rev,
            color)
            _text("\\cdots", Δx, y)
            _arrow(Luxor.Point(Δx + dx_dots, y), Luxor.Point(2Δx - box_width/2, y);
            rev, color)
            _text("$(v)_{K-1}", 2Δx - box_width/2 - dy_up, y - dy_up; color)
            draw_node("$(f)_K", 2Δx, y; stroke_color)
            _text("$(v)_K", 3Δx, y; color)
            _arrow(Luxor.Point(2Δx + box_width/2, y), Luxor.Point(3Δx -
            box_width/2, y); rev, color)
        end
        draw_line("f", "s", -Δy, rev = false, color = colors.s, stroke_color =
        "#546b8c")
        draw_line("\\partial f", "t", 0, rev = false; color = colors.t,
        stroke_color = "#40635c")
        draw_line("\\partial f^*", "r", Δy, rev = true; color = colors.r,
        stroke_color = "#b58b63")
        ks = ["0", "1", "k-1", "k", "K-1", "K"]
        xs = [-3Δx, -2Δx + box_width/2 + dy_up, -box_width/2 - dy_up, box_width/2 +
        dy_up, 2Δx - box_width/2 - dy_up, 3Δx]
        annotate(_text, (; ks, xs, Δy, dy_up, colors))
    end
end
```

fanout_figure

```
fanout_figure(; dup, width = 640, height = 260)
```

Draw the forward and reverse passes for `x` used by both `f_1` and `f_2` with `mathjax_figure`. If `dup` is `true`, `x` is first duplicated with `dup` so that we get a chain.

```
"""
    fanout_figure(; dup, width = 640, height = 260)

Draw the forward and reverse passes for `x` used by both `f_1` and `f_2`
with [`mathjax_figure`](@ref).
If `dup` is `true`, `x` is first duplicated with `dup` so that we get a chain.
"""
function fanout_figure(; dup, width = 640, height = 260)
    mathjax_figure(; width, height) do label
        Luxor = CGE.Luxor
        box_width = 45
        colors = (s = "#7ea6e0", r = "#ffb570", g = "gray")
        function node(text, center; width = box_width)
            Luxor.sethue("white")
            Luxor.box(center, width, box_width, 12, :fill)
            Luxor.sethue("#546b8c")
            Luxor.setline(2)
            Luxor.box(center, width, box_width, 12, :stroke)
            label(text, center.x, center.y)
        end
        # Forward arrow from `p` to `q` and the reverse arrow next to it on
        # the side given by the sign of `side`
        function arrows(p, q; side = 1)
            d = (q - p) / Luxor.distance(p, q)
            n = side * Luxor.Point(d.y, -d.x)
            Luxor.sethue(colors.s)
            Luxor.arrow(p, q; linewidth = 1.5, arrowheadlength = 10)
            Luxor.sethue(colors.r)
            Luxor.arrow(q + 8n, p + 8n; linewidth = 1.5, arrowheadlength = 10)
            return (p + q) / 2, n
        end
        x_f, x_s, Δy = 40, 230, 60
        x_x = dup ? -280 : -180
        label("x", x_x, 0; color = colors.s)
        label("r_0 = {\\color{red}?}", x_x, 28; color = colors.r)
        start = Luxor.Point(x_x + 14, 0)
        if dup
            x_dup = -180
            Luxor.sethue(colors.g)
            Luxor.setline(1)
```

```
            Luxor.setdash("dash")
            Luxor.box(Luxor.Point(x_dup, 0), 80, 70, 8, :stroke)
            Luxor.box(Luxor.Point(x_f, 0), 80, 200, 8, :stroke)
            Luxor.setdash("solid")
            label("g_1", x_dup, -47; color = colors.g)
            label("g_2", x_f, -112; color = colors.g)
            arrows(start, Luxor.Point(x_dup - 25, 0); side = -1)
            node("\\text{dup}", Luxor.Point(x_dup, 0); width = 50)
            start = Luxor.Point(x_dup + 25, 0)
        end
        for (i, side) in [(1, 1), (2, -1)]
            y = (2i - 3) * Δy
            mid, n = arrows(start, Luxor.Point(x_f - box_width / 2, y); side)
            if dup
                p = mid - 12n
                label("x", p.x, p.y; color = colors.s)
            end
            p = mid + 24n
            label("\\partial f_$(i)^* r_$i", p.x, p.y; color = colors.r, fontsize =
            12)
            node("f_$i", Luxor.Point(x_f, y))
            _, n = arrows(Luxor.Point(x_f + box_width / 2, y), Luxor.Point(x_s -
            20, y); side)
            label("s_$i", x_s, y; color = colors.s)
            p = Luxor.Point(x_s, y) + 22n
            label("r_$i", p.x, p.y; color = colors.r)
        end
    end
end
```

```
import LinearAlgebra, DelimitedFiles
```

```
img (generic function with 3 methods)
```

Error message